IND320 – Prosjektarbeid del 2: Datakilder

Student: Khalid Rashed
GitHub (gren `part2-work`): https://github.com/khalidrashed/NMBU-IND320/tree/part2-work
Streamlit-app: https://nmbu-ind320-khalid.streamlit.app/
ENTSO-E-side: https://nmbu-ind320-khalid.streamlit.app/ENTSOE

Formål

Notebooken dokumenterer datakilder og analyse for IND320 del 2: NVE Magasinstatistikk API, ENTSO-E API, Cassandra/Spark, MongoDB og plott. Kjør cellene i rekkefølge. Kontroller at API-parametere, områdekoder og kolonnenavn stemmer med de faktiske responsene dine.

1. Avhengigheter

Installer nødvendige Python-pakker i prosjektets virtuelle miljø. Spark/Cassandra krever i tillegg at connector og lokale tjenester er konfigurert etter kursboken.

In [ ]:
```
# Kjør i terminal ved behov:
# python -m pip install pandas matplotlib requests python-dotenv pymongo
# Spark og Cassandra connector må konfigureres separat.
```

2. Importer biblioteker og last inn miljøvariabler

In [3]:
```
import os
import xml.etree.ElementTree as ET
from datetime import datetime, timedelta, timezone

import pandas as pd
import matplotlib.pyplot as plt
import requests
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
ENTSOE_API_TOKEN = os.getenv("ENTSOE_API_TOKEN")
MONGODB_URI = os.getenv("MONGODB_URI")

print("ENTSO-E token konfigurert:", bool(ENTSOE_API_TOKEN))
print("MongoDB URI konfigurert:", bool(MONGODB_URI))
```

```
ENTSO-E token konfigurert: True
MongoDB URI konfigurert: True
```

3. NVE Magasinstatistikk API

Dokumentasjon:

- NVE Magasinstatistikk API – Swagger
- NVE Magasinstatistikk

Koden under prøver å hente offentlig data fra API-et. Kontroller endepunkt og eventuelle parametere mot Swagger hvis API-et svarer med feil.

In [4]:
```
NVE_URL = "https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/HentOffe

def get_nve_data(url=NVE_URL, params=None, timeout=30):
    response = requests.get(url, params=params, timeout=timeout)
    print("URL:", response.url, "| HTTP:", response.status_code)
    response.raise_for_status()
    payload = response.json()
    if isinstance(payload, list):
        return pd.json_normalize(payload)
    if isinstance(payload, dict):
        list_values = [v for v in payload.values() if isinstance(v, list)]
        return pd.json_normalize(list_values[0] if list_values else payload)
    raise ValueError("Uventet JSON-format")

try:
    nve_df = get_nve_data()
    print("Dimensjon:", nve_df.shape)
    display(nve_df.head())
    print("Kolonner:", nve_df.columns.tolist())
except Exception as exc:
    nve_df = pd.DataFrame()
    print("NVE-kallet må kontrolleres mot Swagger:", type(exc).__name__, exc)
```

```
URL: https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/HentOffentligDa
ta | HTTP: 200
Dimensjon: (14922, 11)
```

	dato_Id	omrType	omrnr	iso_aar	iso_uke	fyllingsgrad	kapasitet_TWh	fylling_TWh
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.913979
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.849344
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.578772
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.082947
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.341054

```
Kolonner: ['dato_Id', 'omrType', 'omrnr', 'iso_aar', 'iso_uke', 'fyllingsgrad',
'kapasitet_TWh', 'fylling_TWh', 'neste_Publiseringsdato', 'fyllingsgrad_forrige_u
ke', 'endring_fyllingsgrad']
```

3.1 Undersøk NVE-kolonner og lag et plott

Velg kolonnenavnene under fra `nve_df.columns`. Ikke gjett: API-responsen kan bruke andre navn. Dette trinnet gjør det mulig å gjenskape plottene fra del 1 med data fra API-et, i stedet for den gamle CSV-filen.

In [5]:
```
if not nve_df.empty:
    display(nve_df.dtypes.to_frame("datatype"))
```

```
    display(nve_df.isna().sum().to_frame("manglende verdier"))

# TODO: erstatt None med faktiske kolonnenavn fra NVE-responsen.
NVE_DATE_COLUMN = None
NVE_VALUE_COLUMN = None
NVE_AREA_COLUMN = None

if not nve_df.empty and NVE_DATE_COLUMN and NVE_VALUE_COLUMN:
    nve_plot = nve_df.copy()
    nve_plot[NVE_DATE_COLUMN] = pd.to_datetime(nve_plot[NVE_DATE_COLUMN], errors
    nve_plot[NVE_VALUE_COLUMN] = pd.to_numeric(nve_plot[NVE_VALUE_COLUMN], error
    nve_plot = nve_plot.dropna(subset=[NVE_DATE_COLUMN, NVE_VALUE_COLUMN])

    if NVE_AREA_COLUMN and NVE_AREA_COLUMN in nve_plot.columns:
        print("Områder:", sorted(nve_plot[NVE_AREA_COLUMN].dropna().astype(str).

    ax = nve_plot.sort_values(NVE_DATE_COLUMN).plot(
        x=NVE_DATE_COLUMN, y=NVE_VALUE_COLUMN, figsize=(12, 5),
        title="NVE Magasinstatistikk – API-data"
    )
    ax.set_xlabel("Dato")
    ax.set_ylabel(NVE_VALUE_COLUMN)
    plt.tight_layout()
    plt.show()
else:
    print("Velg dato- og verdikolonne fra den faktiske NVE-responsen for å lage
```

	datatype
dato_Id	object
omrType	object
omrnr	int64
iso_aar	int64
iso_uke	int64
fyllingsgrad	float64
kapasitet_TWh	float64
fylling_TWh	float64
neste_Publiseringsdato	object
fyllingsgrad_forrige_uke	float64
endring_fyllingsgrad	float64

	manglende verdier
dato_Id	0
omrType	0
omrnr	0
iso_aar	0
iso_uke	0
fyllingsgrad	0
kapasitet_TWh	0
fylling_TWh	0
neste_Publiseringsdato	0
fyllingsgrad_forrige_uke	0
endring_fyllingsgrad	0

```
Velg dato- og verdikolonne fra den faktiske NVE-responsen for å lage plottet.
```

4. ENTSO-E API – siste to år

ENTSO-E API krever en token og riktige EIC-områdekoder. Bekreft parametere og koder i:

- ENTSO-E API reference
- ENTSO-E Transparency Platform

Dette eksemplet henter XML for ett områdepar. Fyll inn faktiske EIC-koder før du kjører det.

In [9]:
```python
ENTSOE_URL = "https://transparency.entsoe.eu/myAccount/webApiAccess"

def fetch_entsoe_flow(token, in_domain, out_domain, start, end, timeout=60):
    if not token:
        raise ValueError("ENTSOE_API_TOKEN mangler i miljøvariablene.")
    params = {
        "securityToken": token,
        "documentType": "A11",
        "in_Domain": in_domain,
        "out_Domain": out_domain,
        "periodStart": start.strftime("%Y%m%d%H%M"),
        "periodEnd": end.strftime("%Y%m%d%H%M"),
    }
    response = requests.get(ENTSOE_URL, params=params, timeout=timeout)
    print("HTTP:", response.status_code)
    response.raise_for_status()
    return response.text

# TODO: fyll inn gyldige EIC-koder fra ENTSO-E-dokumentasjonen.
ENTSOE_AREA_PAIRS = [
    # {"connection": "forbindelsesnavn", "in_domain": "EIC-kode", "out_domain":
]

end_time = datetime.now(timezone.utc).replace(minute=0, second=0, microsecond=0)
start_time = end_time - timedelta(days=730)
```

```
entsoe_xml_responses = {}

if ENTSOE_API_TOKEN and ENTSOE_AREA_PAIRS:
    for pair in ENTSOE_AREA_PAIRS:
        try:
            entsoe_xml_responses[pair["connection"]] = fetch_entsoe_flow(
                ENTSOE_API_TOKEN, pair["in_domain"], pair["out_domain"], start_t
            )
            print(pair["connection"], "mottok XML-data.")
        except Exception as exc:
            print("Feil for", pair["connection"], ":", type(exc).__name__, exc)
else:
    print("ENTSOE_API_TOKEN eller ENTSOE_AREA_PAIRS mangler. Ingen kaller ble gj
```

```
ENTSOE_API_TOKEN eller ENTSOE_AREA_PAIRS mangler. Ingen kaller ble gjort.
```

4.1 XML-parser for ENTSO-E

Parseren under støtter vanlige 15-, 30- og 60-minutters oppløsninger. Kontroller den mot XML-responsen du faktisk får fra ENTSO-E før du bruker dataene videre.

In [10]:
```
def parse_entsoe_xml(xml_text, connection, direction):
    root = ET.fromstring(xml_text)
    namespace = root.tag.split("}")[0].strip("{") if "}" in root.tag else None
    ns = {"e": namespace} if namespace else {}
    prefix = "e:" if namespace else ""

    rows = []
    for series in root.findall(f".//{prefix}TimeSeries", ns):
        period = series.find(f"{prefix}Period", ns)
        if period is None:
            continue
        start_node = period.find(f"{prefix}timeInterval/{prefix}start", ns)
        resolution_node = period.find(f"{prefix}resolution", ns)
        if start_node is None or resolution_node is None:
            continue
        period_start = pd.to_datetime(start_node.text, utc=True, errors="coerce"
        minutes = {"PT15M": 15, "PT30M": 30, "PT60M": 60}.get(resolution_node.te
        if minutes is None:
            print("Ukjent oppløsning:", resolution_node.text)
            continue
        for point in period.findall(f"{prefix}Point", ns):
            pos = point.find(f"{prefix}position", ns)
            qty = point.find(f"{prefix}quantity", ns)
            if pos is None or qty is None:
                continue
            rows.append({
                "timestamp": period_start + pd.Timedelta(minutes=minutes * (int(
                "connection": connection,
                "direction": direction,
                "flow_mw": pd.to_numeric(qty.text, errors="coerce"),
            })
    return pd.DataFrame(rows)

entsoe_frames = []
for pair in ENTSOE_AREA_PAIRS:
    xml_text = entsoe_xml_responses.get(pair["connection"])
    if xml_text:
        try:
```

```
            entsoe_frames.append(parse_entsoe_xml(xml_text, pair["connection"],
        except Exception as exc:
            print("Kunne ikke parse", pair["connection"], ":", exc)

entsoe_api_df = pd.concat(entsoe_frames, ignore_index=True) if entsoe_frames els
print("Rader fra ENTSO-E API:", len(entsoe_api_df))
display(entsoe_api_df.head())
```

```
Rader fra ENTSO-E API: 0
```

5. Cassandra og Spark

Oppgaven krever at ENTSO-E-data settes inn i Cassandra med Spark, og at relevante kolonner hentes ut med Spark. Host, keyspace, tabell, Spark-versjon og Cassandra-connector må passe med oppsettet ditt fra kursboken. Eksemplet under er med vilje kommentert ut til du har bekreftet disse innstillingene.

In [ ]:
```
# Eksempel – tilpass versjoner og navn til din installasjon:
#
# from pyspark.sql import SparkSession
# spark = (
#     SparkSession.builder
#     .appName("IND320-ENTSOE")
#     .config("spark.cassandra.connection.host", os.getenv("CASSANDRA_HOST", "12
#     # .config("spark.jars.packages", "com.datastax.spark:spark-cassandra-conne
#     .getOrCreate()
# )
#
# if not entsoe_api_df.empty:
#     spark_df = spark.createDataFrame(entsoe_api_df)
#     (spark_df.write.format("org.apache.spark.sql.cassandra")
#      .options(keyspace=os.getenv("CASSANDRA_KEYSPACE", "ind320"),
#               table=os.getenv("CASSANDRA_TABLE", "entsoe_crossborder_flows"))
#      .mode("append").save())
#
# # Hent relevante kolonner tilbake fra Cassandra:
# cassandra_df = (
#     spark.read.format("org.apache.spark.sql.cassandra")
#     .options(keyspace=os.getenv("CASSANDRA_KEYSPACE", "ind320"),
#              table=os.getenv("CASSANDRA_TABLE", "entsoe_crossborder_flows"))
#     .load().select("timestamp", "connection", "direction", "flow_mw")
# )
# cassandra_df.show(10, truncate=False)
# entsoe_spark_df = cassandra_df.toPandas()
```

6. Les klargjorte ENTSO-E-data fra MongoDB

Denne delen bruker samlingen som Streamlit-siden din allerede forventer: `ind320.entsoe_crossborder_flows`. Dette gir et konkret datasett for plott. Det erstatter ikke dokumentasjon av Cassandra/Spark dersom de stegene ennå ikke er kjørt.

In [12]:
```
def load_entsoe_from_mongodb(uri):
    if not uri:
        raise ValueError("MONGODB_URI mangler i miljøvariablene.")
    client = MongoClient(uri, serverSelectionTimeoutMS=10000)
    try:
```

```
        collection = client["ind320"]["entsoe_crossborder_flows"]
        projection = {
            "_id": 0, "timestamp": 1, "connection": 1,
            "from_area": 1, "to_area": 1, "direction": 1, "flow_mw": 1,
        }
        return pd.DataFrame(collection.find({}, projection))
    finally:
        client.close()

try:
    entsoe_df = load_entsoe_from_mongodb(MONGODB_URI)
    if not entsoe_df.empty:
        entsoe_df["timestamp"] = pd.to_datetime(entsoe_df["timestamp"], errors="
        entsoe_df["flow_mw"] = pd.to_numeric(entsoe_df["flow_mw"], errors="coerc
        entsoe_df = entsoe_df.dropna(subset=["timestamp", "connection", "directi
    print("MongoDB-dimensjon:", entsoe_df.shape)
    display(entsoe_df.head())
except Exception as exc:
    entsoe_df = pd.DataFrame()
    print("Kunne ikke lese MongoDB:", type(exc).__name__, exc)
```

```
MongoDB-dimensjon: (945264, 6)
```

	timestamp	to_area	connection	from_area	direction	flow_mw
0	2024-10-14 22:00:00+00:00	SE_3	NO_1-SE_3	NO_1	from_Norway	0.0
1	2024-10-14 23:00:00+00:00	SE_3	NO_1-SE_3	NO_1	from_Norway	0.0
2	2024-10-15 00:00:00+00:00	SE_3	NO_1-SE_3	NO_1	from_Norway	0.0
3	2024-10-15 01:00:00+00:00	SE_3	NO_1-SE_3	NO_1	from_Norway	0.0
4	2024-10-15 02:00:00+00:00	SE_3	NO_1-SE_3	NO_1	from_Norway	0.0

7. Plott 1 – årlig import og eksport for en valgt forbindelse

Effekt måles i MW, mens energi måles i MWh. Estimatet beregner MW × antall timer til neste observasjon. Intervaller over én time ekskluderes fordi de kan inneholde manglende målinger. Tolk derfor kakediagrammet som et estimat, ikke et offisielt årsresultat.

In [13]:
```
if not entsoe_df.empty:
    plot_df = entsoe_df.copy()
    plot_df["year"] = plot_df["timestamp"].dt.year
    connections = sorted(plot_df["connection"].dropna().unique().tolist())
    years = sorted(plot_df["year"].dropna().unique().tolist())
    print("Forbindelser:", connections)
    print("År:", years)

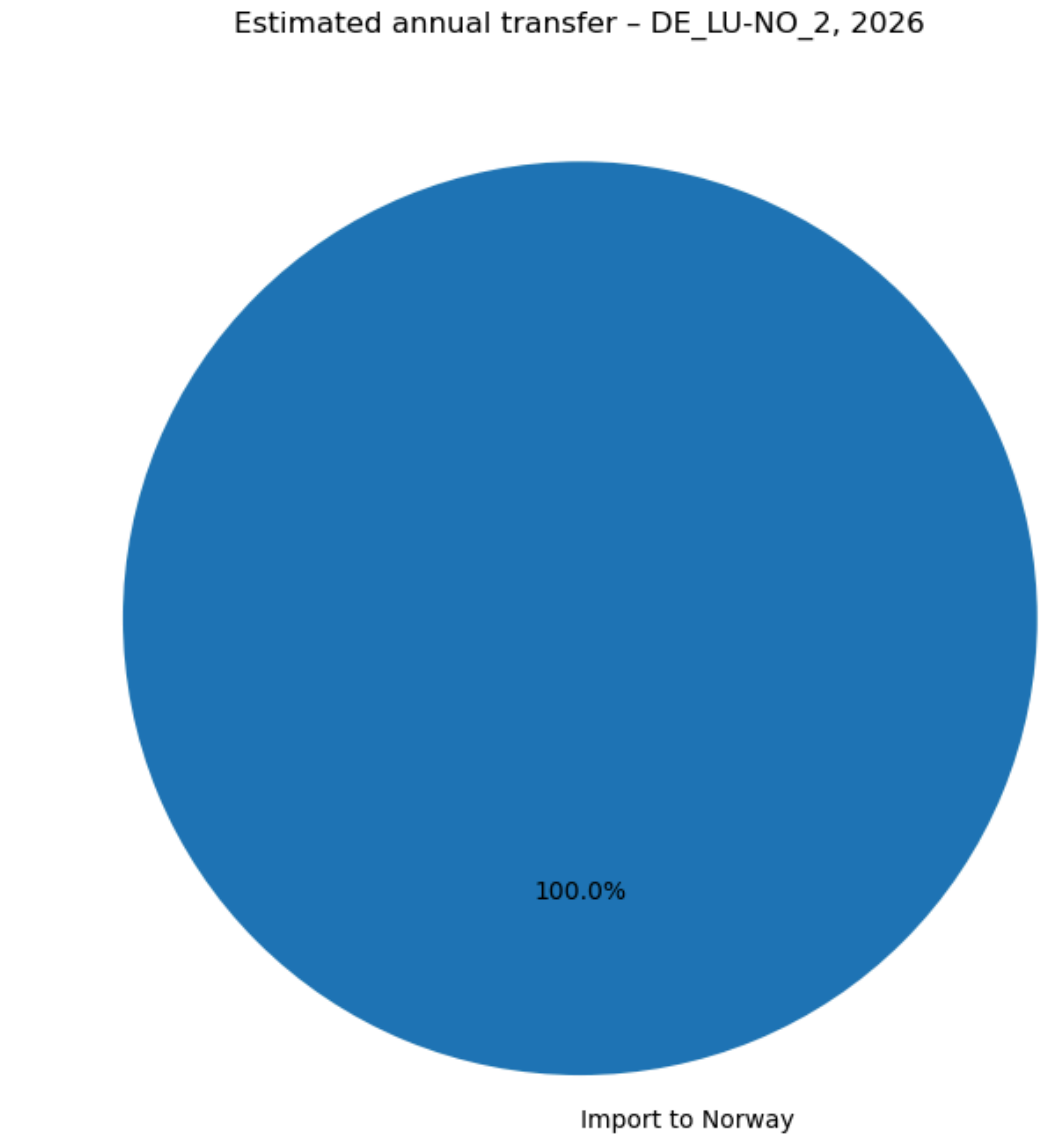
    # Standardvalg. Endre etter å ha kontrollert listene ovenfor.
    SELECTED_CONNECTION = connections[0]
    SELECTED_YEAR = years[-1]

    annual = plot_df[(plot_df["connection"] == SELECTED_CONNECTION) &
                     (plot_df["year"] == SELECTED_YEAR)].copy()
    annual = annual.sort_values(["direction", "timestamp"])
    annual["next_timestamp"] = annual.groupby("direction")["timestamp"].shift(-1
    annual["interval_hours"] = (annual["next_timestamp"] - annual["timestamp"]).
    annual["interval_hours"] = annual["interval_hours"].where(annual["interval_h
```

```
    annual["estimated_mwh"] = annual["flow_mw"].abs() * annual["interval_hours"]
    energy = annual.dropna(subset=["estimated_mwh"]).groupby("direction", as_ind

    labels = {"to_Norway": "Import to Norway", "from_Norway": "Export from Norwa
    if not energy.empty and energy["estimated_mwh"].sum() > 0:
        energy["label"] = energy["direction"].map(lambda x: labels.get(x, x))
        plt.figure(figsize=(7, 7))
        plt.pie(energy["estimated_mwh"], labels=energy["label"], autopct="%1.1f%
        plt.title(f"Estimated annual transfer – {SELECTED_CONNECTION}, {SELECTED
        plt.tight_layout()
        plt.show()
        display(energy)
    else:
        print("Ingen gyldige energiintervaller for valgt forbindelse og år.")
else:
    print("MongoDB-data mangler; kontroller URI og at samlingen inneholder data.
```

```
Forbindelser: ['DE_LU-NO_2', 'DK_1-NO_2', 'FI-NO_4', 'GB-NO_2', 'NL-NO_2', 'NO_1-
SE_3', 'NO_2-DE_LU', 'NO_2-DK_1', 'NO_2-GB', 'NO_2-NL', 'NO_3-SE_2', 'NO_4-FI',
'NO_4-SE_1', 'SE_1-NO_4', 'SE_2-NO_3', 'SE_3-NO_1']
År: [2024, 2025, 2026]
```



	direction	estimated_mwh	label
0	to_Norway	2.251446e+06	Import to Norway

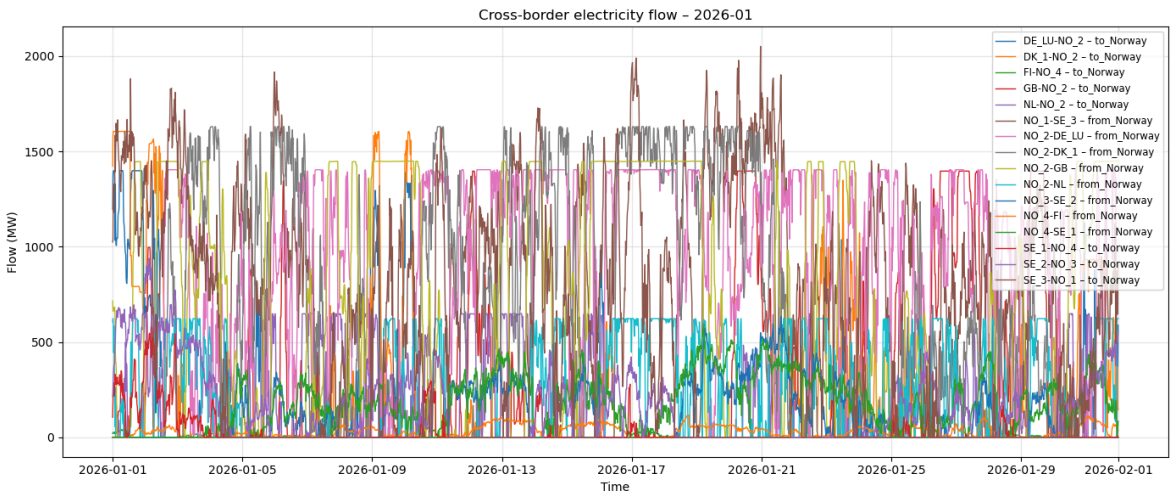
8. Plott 2 – linjediagram for en måned

Hver linje representerer en kombinasjon av forbindelse og retning. Velg år og måned etter at du har sett tilgjengelige data.

In [14]:
```
if not entsoe_df.empty:
    plot_df = entsoe_df.copy()
    plot_df["year"] = plot_df["timestamp"].dt.year
    plot_df["month"] = plot_df["timestamp"].dt.month

    SELECTED_YEAR_LINE = int(plot_df["year"].max())
    SELECTED_MONTH = 1  # januar

    month_df = plot_df[(plot_df["year"] == SELECTED_YEAR_LINE) &
                       (plot_df["month"] == SELECTED_MONTH)].copy()
    if not month_df.empty:
        month_df["series"] = month_df["connection"].astype(str) + " - " + month_
        fig, ax = plt.subplots(figsize=(14, 6))
        for label, group in month_df.groupby("series"):
            group = group.sort_values("timestamp")
            ax.plot(group["timestamp"], group["flow_mw"], label=label, linewidth
        ax.set_title(f"Cross-border electricity flow - {SELECTED_YEAR_LINE}-{SEL
        ax.set_xlabel("Time")
        ax.set_ylabel("Flow (MW)")
        ax.legend(loc="best", fontsize="small")
        ax.grid(True, alpha=0.3)
        fig.tight_layout()
        plt.show()
    else:
        print("Ingen data for valgt måned og år.")
else:
    print("MongoDB-data mangler; linjediagrammet kan ikke lages.")
```



9. Datakvalitet

Bruk de faktiske resultatene fra denne cellen i beskrivelsen din. Ikke oppgi tall før du har kjørt notebooken.

In [15]:
```python
if not entsoe_df.empty:
    print("Antall rader:", len(entsoe_df))
    print("Antall forbindelser:", entsoe_df["connection"].nunique())
    print("Retninger:", sorted(entsoe_df["direction"].dropna().unique().tolist())
    print("Første tidsstempel:", entsoe_df["timestamp"].min())
    print("Siste tidsstempel:", entsoe_df["timestamp"].max())
    display(entsoe_df.isna().sum().to_frame("Manglende verdier"))
else:
    print("Ingen data tilgjengelig for kvalitetskontroll.")
```

```
Antall rader: 945264
Antall forbindelser: 16
Retninger: ['from_Norway', 'to_Norway']
Første tidsstempel: 2024-10-14 22:00:00+00:00
Siste tidsstempel: 2026-10-14 21:45:00+00:00
```

	Manglende verdier
timestamp	0
to_area	0
connection	0
from_area	0
direction	0
flow_mw	0

10. Oppsummering

1. NVE Magasinstatistikk skal hentes direkte fra API-et og brukes til å gjenskape plottene fra del 1.
2. ENTSO-E skal levere grensekryssende flytdata for de siste to årene.
3. Spark brukes til å lagre data i Cassandra og hente relevante kolonner.
4. Klargjorte data lagres i MongoDB.
5. Streamlit leser MongoDB-data og viser årsdiagram og månedlig linjediagram.

11. Beskrivelse av AI-bruk

Jeg brukte ChatGPT som støtte til å forstå feilmeldinger, strukturere Python-kode, feilsøke pakkeinstallasjoner og forbedre dokumentasjonen. Kodeforslag fra AI ble behandlet som utgangspunkt og må testes mot prosjektets faktiske API-responser, databaseoppsett og datastruktur. Jeg kontrollerte arbeidet ved å kjøre kode lokalt, undersøke kolonner og teste Streamlit-applikasjonen. Hemmeligheter som API-token og databasepassord skal lagres i miljøvariabler eller Streamlit Secrets, ikke i notebooken eller på GitHub.

12. Arbeidslogg (utkast, 300–500 ord)

I del 2 av IND320 arbeidet jeg med å koble sammen eksterne datakilder, databaser og en Streamlit-applikasjon. Målet var å bruke API-er i stedet for å være avhengig av en lokal CSV-fil, og å gjøre dataene tilgjengelige både for analyse i Jupyter Notebook og for visning på nett.

Jeg undersøkte NVE sitt Magasinstatistikk API og arbeidet med å hente data direkte fra API-et. En viktig del av arbeidet er å kontrollere hvilke tidskolonner, områdekoder og måleverdier som finnes i responsen før dataene renses og plottes. API-data kan ha andre feltnavn eller formater enn en CSV-fil, så datoer og manglende verdier må håndteres eksplisitt. Målet er å gjenskape relevante figurer fra del 1 med data som hentes direkte fra NVE.

Jeg arbeidet også med ENTSO-E API for grensekryssende kraftflyt til og fra Norge. Dataene skal dekke de siste to årene. API-et krever en sikkerhetstoken og riktige områdekoder, og responsen må parses før dataene kan brukes videre. Dataflyten for oppgaven innebærer å bruke Spark til å lagre data i Cassandra og hente ut relevante kolonner. De klargjorte dataene lagres deretter i MongoDB slik at Streamlit-applikasjonen kan hente dem.

I Streamlit arbeidet jeg med en egen ENTSO-E-side med to kolonner. Den ene viser en årlig sammenligning av import og eksport for en valgt forbindelse, mens den andre viser kraftflyt over valgte måneder. Diagrammene bruker MW-observasjoner. Når årlig energi estimeres i MWh, må varigheten mellom observasjonene tas med i beregningen. Manglende eller lange intervaller kan påvirke estimatet, og dette må forklares for brukeren.

Jeg brukte ChatGPT som støtte for feilsøking, kodeforslag og dokumentasjon. Jeg måtte teste forslagene i mitt eget miljø, blant annet fordi Python-avhengigheter og konfigurasjon kan gi feil. Jeg lærte at hemmeligheter må lagres som miljøvariabler eller i Streamlit Secrets, og ikke i offentlig kode. Før innlevering må jeg kontrollere at alle celler kjører i riktig rekkefølge, verifisere API-parametere og områdekoder, kontrollere resultatene og eksportere notebooken til PDF.

13. Referanser og prosjektlenker

- NVE Magasinstatistikk API – Swagger
- NVE Magasinstatistikk
- ENTSO-E API reference
- ENTSO-E Transparency Platform
- GitHub – part2-work
- Streamlit-app
- ENTSO-E-side